♠ ♥ ♦ ♣

ANALISI OOP

Il Gioco del Blackjack

Esercizi di analisi orientata agli oggetti in C++

Come usare questo documento

Questo documento ti guida nell'analisi del gioco del Blackjack usando i concetti della Programmazione Orientata agli Oggetti (OOP) che hai studiato. Non si tratta di scrivere il codice completo del gioco, ma di allenare il tuo pensiero da progettista: capire quali classi servono, cosa devono sapere e cosa devono fare.

Per ogni sezione troverai:

- Una breve descrizione della competenza OOP affrontata
- Il modello delle classi del Blackjack come punto di riferimento
- Domande aperte da rispondere con ragionamenti motivati
- Una traccia (hint) per orientarti se sei in difficolta

Attenzione alle domande trabocchetto (segnalate con il simbolo ⚠): sono domande con una risposta intuitiva ma errata. Scopri perche!

♠ Il Blackjack in poche parole

Leggi attentamente: da questa descrizione nascono le classi

Il Blackjack e' un gioco di carte tra uno o piu' Giocatori e un Mazziere. Avvicinarsi il più possibile a 21 punti senza superarli, ottenendo un punteggio più alto del Mazziere.

All'inizio di ogni round il Mazziere distribuisce due carte a ciascun Giocatore e due a se stesso (una delle sue carte rimane coperta). Ogni Giocatore puo' chiedere un'altra carta ("pesca") oppure fermarsi. Se supera 21, ha perso ("sballato"). Alla fine, il Mazziere scopre la sua carta nascosta e pesca fino a raggiungere almeno 17. Chi si avvicina di piu' a 21 senza sballare vince.

Valore delle carte

Carta	Valore	Esempio
Carte numeriche	Valore stampato	7 di Cuori = 7
Figure (J, Q, K)	10	Re di Picche = 10
Asso	1 oppure 11	Sceglie il valore più favorevole

Azioni del Giocatore

- **Hit (Chiedi carta)** — pesca un'altra carta dal Mazzo.
- **Stand (Stai)** — fine del tuo turno, passa al Mazziere.
- **Double (Raddoppia)** — raddoppia la puntata, pesca esattamente una carta e stai.

Glossario Minimo

Termine	Significato
Mano	Le carte attualmente in possesso di un giocatore o del Mazziere.
Mazzo	Il mazzo di 52 carte da cui si pescano le carte durante la partita.
Puntata	L'importo di crediti che il Giocatore mette in gioco prima di ogni mano.
Bust	Sballato: punteggio superiore a 21. Chi va bust perde immediatamente.
Blackjack	Asso + carta da 10 come prime due carte. Vittoria immediata se non pareggiata dal Mazziere.

Le classi del Blackjack — il modello di riferimento

Queste sono le classi che useremo come riferimento per tutti gli esercizi. Studialo bene prima di rispondere alle domande.

Responsabilità delle Classi

Ogni classe ha una responsabilità precisa. La tabella mostra cosa contiene e cosa sa fare ciascuna.

Classe	Contiene	Sa fare
Carta	Valore (1–13) Seme (Cuori, Quadri, Fiori, Picche) Stato: coperta / scoperta	Dire il proprio valore base nel Blackjack Passare da coperta a scoperta e viceversa
Mazzo	Array di 52 oggetti Carta	Inizializzarsi con tutte le 52 carte Mescolare le carte in ordine casuale Consegnare la prima carta disponibile (pesca)
Mano	Array di Carta (max 12) Numero di carte presenti	Aggiungere una carta ricevuta Calcolare il punteggio (gestisce il doppio valore dell'Asso) Sapere se è sballata (bust) o se è Blackjack
Giocatore	Mano Nome Budget Puntata corrente	Ricevere una carta (la aggiunge alla sua Mano) Impostare e gestire la puntata Aggiornare il budget dopo ogni mano
Mazziere	Mano Una carta inizialmente coperta	Ricevere una carta Scoprire la carta coperta al momento giusto Decidere autonomamente se pescare (regola: sotto 17 pesca sempre)
GameEngine	Mazzo Giocatore Mazziere Stato del turno e dell'esito	Avviare una nuova mano Passare le carte da Mazzo a Giocatore o Mazziere Eseguire il turno del Mazziere Determinare e applicare il risultato (vince, perde, pareggio)
IOTestuale	Campo Tastiera LettoreInput	Mostrare lo stato della partita a schermo Chiedere nome, puntata e azione all'utente Mostrare i messaggi di esito

Relazioni tra le Classi

Tabella delle relazioni

Due tipi di relazione:

- **Possiede**: l'oggetto B fa parte di A e viene distrutto insieme a lui.
- **Descrizione dell'Atto d'Uso**

Chi	Relazione	Con chi	Osservazioni
GameEngine	**Possiede e Usa**	**Mazzo**	*GameEngine possiede un solo mazzo per tutta la partita e lo usa per pescare dal Mazzo e ricevere un oggetto Carta o per rimescolarlo.*
GameEngine	**Possiede e Usa**	**Giocatore**	*GameEngine possiede un solo giocatore umano e lo usa per impostare la puntata scelta dall'utente o aggiungere una carta alla mano.*
GameEngine	**Possiede e Usa**	**Mazziere**	*GameEngine possiede il banco e lo usa per sapere se deve pescare ancora o per chiedere di scoprire la carta coperta.*
Giocatore	**Possiede e Usa**	**Mano**	*Giocatore possiede la mano che viene svuotata e ricreata ad ogni mano Per delegare il calcolo del punteggio e capire se è "Sballato".*
Mazziere	**Possiede e Usa**	**Mano**	*Come Giocatore, ma segue regole diverse e per verificare la regola del 17.*
Mano	**Possiede**	**Carta (0..*)**	*Mano possiede da 0 a 12 carte per volta*
Mazzo	**Possiede**	**Carta (52)**	*Mazzo all'inizializzazione possiede sempre 52 carte*
IOTestuale	**Possiede e Usa**	**Campo**	*IOTestuale possiede un campo da gioco per vsualizzare l'output su terminale*
IOTestuale	**Possiede e Usa**	**Tastiera**	*IOTestuale possiede una Tastiera per leggere i tasti premuti dall'utente*
IOTestuale	**Possiede e Usa**	**LettoreInput**	*IOTestuale possiede un LettoreInput per fare domande e ricevere risposte*
IOTestuale	**Usa**	**GameEngine**	*IOTestuale legge lo stato della partita da GameEngine per visualizzarlo*
GameEngine	**Usa**	**IOTestuale**	*GameEngine usa IOTestuale sia per l'output che per le richieste di input.*

Ciclo di una mano (5 passi)

1. **Inizio mano:** GameEngine chiede a VistaTestuale la puntata, poi svuota le mani e rimescola il Mazzo se necessario.
2. **Distribuzione:** GameEngine pesca dal Mazzo 2 carte per il Giocatore e 2 per il Mazziere (una di queste coperta).
3. **Turno del Giocatore:** VistaTestuale chiede l'azione (Hit / Stand / Double), GameEngine la esegue finché il Giocatore non si ferma o va bust.
4. **Turno del Mazziere:** GameEngine scopre la carta coperta, poi chiede al Mazziere se deve pescare; il Mazziere pesca finché il suo punteggio è inferiore a 17.
5. **Esito:** GameEngine confronta i punteggi, aggiorna il budget del Giocatore e VistaTestuale mostra il messaggio finale.

Scelte di Progetto

Perché tre classi per l'input/output?

Nella versione attuale il gioco funziona sul terminale: Campo usa cout, Tastiera usa cin. Potrebbe sembrare eccessivo usare tre classi per qualcosa di così semplice.

La ragione è la seguente: se in futuro si vorrà aggiungere la grafica con SFML, sarà necessario sostituire cout e cin con la gestione di una finestra e degli eventi. Se queste istruzioni fossero sparse nel codice, bisognerebbe cercarle e riscriverle ovunque.

Con questa architettura invece:

- **Campo** — definisce dove e come si mostra qualcosa (oggi: terminale, domani: finestra SFML).
- **Tastiera** — definisce come si legge un tasto (oggi: cin, domani: sf::Event di SFML).
- **LettoreInput** — coordina la visualizzazione di una domanda e la raccolta della risposta. Se utilizza Campo e Tastiera non cambia, indipendentemente dalla tecnologia usata.

> Quando si aggiungerà la grafica, GameEngine, Carta, Mano, Mazzo, Giocatore e Mazziere non cambieranno la parte logica, ma solo il loro aspetto grafico. Si riscriveranno le classi Campo, Tastiera ed eventualmente LettoreInput perché sono loro le uniche classi a sapere come funziona l’input e l'output.

Perché GameEngine conosce tutto?

GameEngine possiede Mazzo, Giocatore e Mazziere ed è l'unico a gestire i turni e l'esito. Questo lo rende una classe con molte responsabilità.

È una scelta consapevole per questa fase del progetto: permette di avere un unico punto di controllo facile da seguire. Quando il progetto crescerà, si potrà scomporre GameEngine in classi più piccole, ciascuna con una sola responsabilità. Riconoscere quel momento è una delle lezioni che si imparano sviluppando.

1 Le Classi — Il Progetto

Saper distinguere tra il modello astratto e l'entita' concreta

Una classe e' lo stampo, non l'oggetto. Carta e' la classe; il 7 di cuori che stai tenendo in mano e' l'oggetto (l'istanza). Quando progetti, devi sempre chiederti: "Sto descrivendo una categoria di cose, o una cosa specifica?"

Domanda 1 *[Classe]*

Nel Blackjack, uno studente propone di creare una classe chiamata CartaDiCuori e una chiamata CartaDiFiori, invece di una singola classe Carta. Dal punto di vista della progettazione OOP, questa scelta e' corretta? Motiva la risposta spiegando cosa si perde e cosa si potrebbe fare invece.

Traccia: *Pensa a quante classi dovresti creare per coprire tutti i semi e tutti i valori. Cosa cambia tra una Carta di cuori da 7 e una di fiori da 7?*

Domanda 2 *[Classe]*

Nel modello qui sopra non compare una classe chiamata Round o Turno. Secondo te andrebbe aggiunta? Descrivi cosa rappresenterebbe (cosa 'sa' e cosa 'fa') e se la sua assenza e' un problema per il funzionamento del gioco.

Traccia: *Prova a elencare le azioni che avvengono in un round. Queste azioni appartengono gia' a qualche classe esistente o non hanno un 'proprietario'?*

Domanda 3 *[Classe]*

Un tuo compagno sostiene che Mazziere e Giocatore sono la stessa cosa e che basta una sola classe Partecipante. Ha ragione? Elenca almeno due differenze di comportamento o di stato che giustificano due classi distinte.

Traccia: *Guarda il modello: Mazziere ha un attributo che Giocatore non ha. E i metodi? Ci sono azioni che puo' fare solo il Mazziere?*

Domanda 4 *[Classe]*

Perché Mano e Mazzo sono classi diverse?

Traccia: Il Mazzo è l'insieme di tutte le 52 carte: esiste uno solo. I metodi sono diversi?

.

2 Gli attributi — Lo Stato

Saper identificare quali dati definiscono l'identita' di un oggetto

Gli attributi definiscono cosa un oggetto 'sa' di se stesso. La domanda giusta non e' 'dove posso mettere questo dato?' ma 'a chi appartiene questa informazione per natura?'

Domanda 4 *[Attributo]*

Se il Mazziere deve tenere una carta coperta, quale attributo booleano aggiungeresti e a quale classe lo assegneresti per garantire che il punteggio totale rimanga segreto all'utente? Scrivi la dichiarazione C++ dell'attributo e motiva la scelta della classe.

Traccia: *Il fatto che una carta sia coperta e' una proprieta' della carta stessa o della situazione del Mazziere? Ci possono essere altri contesti in cui una carta e' coperta?*

Domanda 5 *[Attributo]*

La classe Mano contiene un array di Carta e un intero numCarte. Perche' e' necessario mantenere entrambi? Non basterebbe scorrere l'array ogni volta per contare le carte? Motiva la risposta considerando anche la leggibilita' e l'efficienza del codice.

Traccia: *Cosa succede se devi chiamare hasSballato() decine di volte per turno? Cosa cambia tra 'ricalcolare' e 'ricordare'?*

Domanda 6 *[Attributo]*

Uno studente aggiunge un attributo punteggioTotale alla classe Mano, da aggiornare ogni volta che viene aggiunta una carta. Un altro studente dice che e' meglio calcolarlo al momento con calcolaPunteggio(). Quale delle due scelte ti sembra piu' corretta per il Blackjack, dove l'Asso puo' valere 1 o 11? Argomenta.

Traccia: *Nel Blackjack il valore di un Asso dipende dalle altre carte in mano. Se salvi il punteggio come attributo, quando lo aggiorni? E se arriva un'altra carta?*

Domanda 7 *[Attributo]*

Perche' nella classe Carta il seme e' dichiarato come string e non come int (ad esempio 0=cuori, 1=quadri, 2=fiori, 3=picche)? Ci sono vantaggi e svantaggi in entrambe le scelte? Descrivi almeno uno per parte.

Traccia: *Cosa succede quando devi stampare il seme a schermo? E se devi confrontare due semi?*

3 I metodi — Il Comportamento

Valutare la capacita' di ripartire le azioni in blocchi logici

Un metodo e' una responsabilita' ben definita. Deve fare una cosa sola, farla bene, e il suo nome deve dire esattamente cosa fa. Se per spiegare un metodo usi la parola 'e', probabilmente dovrebbe essere diviso in due.

Domanda 8 *[Metodo]*

Descrivi la dichiarazione e il comportamento del metodo distribuisci() della classe Mazzo. Cosa deve restituire? Come deve cambiare lo stato interno del Mazzo dopo la chiamata? Cosa dovrebbe fare il metodo se il mazzo e' esaurito?

Traccia: *Un mazzo fisico si 'svuota' man mano che distribuisci. Come tieni traccia di quante carte hai gia' distribuito? L'attributo indicePesca ti da' un indizio.*

Domanda 9 *[Metodo]*

Il metodo calcolaPunteggio() nella classe Mano deve gestire il caso speciale dell'Asso (vale 1 o 11). Descrivi a parole la logica che useresti: come decidi se l'Asso vale 1 o 11? Ci possono essere piu' Assi nella stessa mano?

Traccia: *Inizia contando tutti gli Assi come 11. Poi controlla se il totale supera 21. In quel caso, cosa fai con uno degli Assi?*

Domanda 10 *[Metodo]*

Un metodo chiamato gestisciTurnoGiocatore() nella classe Giocatore fa queste cose: mostra la mano, chiede all'utente se pescare o fermarsi, pesca la carta dal mazzo, aggiunge la carta alla mano e controlla se ha sballato. E' un buon metodo? Identificane i problemi e proponi come suddividerlo.

Traccia: *Conta quante 'responsabilita" diverse ha questo metodo. Poi chiediti: quale classe dovrebbe occuparsi di ciascuna di esse?*

Domanda 11 *[Metodo]*

Il metodo hasSballato() di Mano deve restituire true se il punteggio supera 21. Ha senso che questo metodo chiami internamente calcolaPunteggio(), oppure dovrebbe ricevere il punteggio come parametro? Confronta le due versioni.

Traccia: *Se hasSballato() chiama calcolaPunteggio() internamente, cosa cambia per chi usa la classe Mano? Chi deve 'sapere' come si calcola il punteggio?*

4 L'incapsulamento — Protezione e Interfaccia

Verificare se si sa proteggere l'integrita' dei dati

Incapsulare non significa solo mettere 'private' davanti a tutto. Significa decidere consapevolmente quale parte del tuo oggetto e' visibile all'esterno (l'interfaccia pubblica) e quale e' un dettaglio interno che nessun altro deve toccare direttamente.

Domanda 12 *[Incapsulamento]*

Nella classe Mazzo, l'array carte e' private. Un tuo compagno dice: 'basterebbe renderlo public, cosi' chiunque puo' accedere alle carte direttamente e risparmiamo i metodi'. Spiega almeno due problemi concreti che nascerebbero nel gioco del Blackjack se si seguisse questo consiglio.

Traccia: *Cosa succederebbe se il codice che gestisce il turno del giocatore potesse leggere tutte le carte del mazzo in anticipo? E se potesse cambiarle?*

Domanda 13 *[Incapsulamento]*

La classe Mano ha il metodo mostra() che stampa le carte. Perche' e' meglio che questo metodo stia nella classe Mano piuttosto che scorrere l'array di carte dall'esterno e stamparle una a una nel main?

Traccia: *Cosa succede se in futuro vuoi cambiare il formato di stampa (ad esempio aggiungere colori o simboli)? In quanti posti del codice dovresti fare modifiche?*

⚠ **Domanda trabocchetto** *[Incapsulamento]*

La classe Mazziere ha cartaCoperta come attributo private. Il metodo scopriCarta() cambia questo valore a false. Ma il punteggio del Mazziere e' calcolato da calcolaPunteggio() in Mano, che non sa se una carta e' coperta. C'e' un problema di coerenza? Come lo risolveresti?

Traccia: *Attenzione: cartaCoperta e' un attributo del Mazziere, ma le carte stanno nella Mano. Chi dei due dovrebbe 'nascondere' il contributo al punteggio di una carta coperta?*

5 La composizione — Relazione "Ha un"

Valutare la capacita' di costruire oggetti complessi assemblando pezzi semplici

La composizione e' il modo principale in cui gli oggetti collaborano: un oggetto 'contiene' un altro oggetto come attributo. Giocatore HA UNA Mano. Mano HA N Carte. Mazzo HA 52 Carte. Costruire bene la composizione significa evitare oggetti troppo grandi e responsabilita' duplicate.

Domanda 15 *[Composizione]*

Sia Giocatore che Mazziere hanno un attributo mano di tipo Mano. Questo significa che stanno 'duplicando' la logica del punteggio? Oppure e' la scelta corretta? Giustifica la tua risposta usando il concetto di composizione.

Traccia: *Duplicare codice e duplicare la struttura sono due cose diverse. Giocatore e Mazziere hanno entrambi una Mano, ma chi implementa la logica del punteggio?*

Domanda 16 *[Composizione]*

Se decidessi di aggiungere un mazzo di 'Carte Speciali' (es. una carta Jolly che raddoppia il premio), modificheresti la classe Carta esistente o ne creeresti una nuova? Motiva la scelta in base alla flessibilita' del codice e alla composizione delle classi.

Traccia: *Cosa succede alla classe Mazzo se aggiungi proprieta' speciali direttamente in Carta? Tutte le 52 carte del mazzo standard avrebbero senso con quei nuovi attributi?*

⚠ **Domanda trabocchetto** *[Composizione]*

La classe Mazzo contiene 52 oggetti Carta in un array. Potresti costruire il Mazzo senza usare la composizione, memorizzando solo i valori interi (es. 1-52) e calcolando seme e valore ogni volta. Quali sono i vantaggi e gli svantaggi di questa alternativa rispetto alla composizione?

Traccia: *Pensa alla leggibilita' del codice e a cosa succede quando aggiungi funzionalita': con numeri interi, dove metti il metodo isPictura()?*

6 Le responsabilita' — Chi fa cosa

Capire se si sa evitare la creazione di 'Classi Dio' che fanno tutto

Ogni classe deve avere una sola ragione di esistere e una responsabilità chiara. Se una classe conosce troppo degli altri, o fa cose che non le competono, diventa fragile: ogni modifica al gioco richiede di toccarla. La domanda da farsi sempre e': 'Questa azione appartiene davvero a questa classe?'

Domanda 18 *[Responsabilità]*

Immagina che uno studente crei una sola classe Blackjack che gestisce tutto: il mazzo, le mani, i turni, il punteggio, la grafica e le regole. Quali sono i problemi pratici di questa scelta quando il gioco deve essere modificato (es. aggiungere un nuovo tipo di giocatore o cambiare le regole del Mazziere)?

Traccia: *Prova a pensare a cosa devi toccare nella classe Blackjack ogni volta che cambia anche solo una piccola regola. E se vuoi riusare il Mazzo in un altro gioco di carte?*

Domanda 19 *[Responsabilità]*

Il metodo deveRiPescare() nella classe Mazziere controlla se il punteggio del Mazziere e' inferiore a 17. Questo metodo potrebbe stare anche in una classe chiamata RegleDelGioco. Quale delle due posizioni ti sembra piu' corretta e perche'? Ci sono argomenti validi per entrambe?

Traccia: *Chi 'possiede' la regola del 17? E' una caratteristica intrinseca del Mazziere o e' una regola esterna che qualcuno gli impone?*

⚠ **Domanda trabocchetto** *[Responsabilità]*

Nella classe Mano c'e' il metodo mostra() che stampa le carte a schermo. E' corretto che una classe che rappresenta 'dati di gioco' si occupi anche della stampa su schermo? Proponi un'alternativa e discutine i pro e i contro.

Traccia: *Cosa succede se vuoi usare le stesse classi in un'interfaccia grafica invece che in un terminale? La logica di gioco e la logica di visualizzazione sono la stessa cosa?*

Domanda 21 *[Responsabilità]*

Scrivi la dichiarazione C++ (solo l'intestazione, non l'implementazione) di una classe Partita che abbia la responsabilità di coordinare un round del gioco. Quali attributi deve avere? Quali metodi? Giustifica ogni scelta.

Traccia: *Una classe Partita non deve implementare le regole di Carta o Mano: deve coordinarle. Pensa a cosa 'sa' una partita (quali oggetti contiene) e cosa 'fa' (i passi di un round).*

Riepilogo delle competenze

1	Classe	Saper distinguere il modello astratto dall'entita' concreta
2	Attributo	Identificare i dati che definiscono l'identita' di un oggetto
3	Metodo	Ripartire le azioni in blocchi logici con responsabilita' chiara
4	Incapsulamento	Proteggere l'integrita' dei dati e progettare l'interfaccia
5	Composizione	Costruire oggetti complessi assemblando pezzi semplici
6	Responsabilita'	Distribuire bene le responsabilita', evitare le Classi Dio